Lab sheet :- 6

Queue programs (short versions)

Qu1. Queue using array (Enqueue, Dequeue, peek, display)

```
#include <iostream>
using name space
int q(5), f = 0, r = -1;

void enqueue (int x) [if (r = -1) cout << "overflow\n"; else
q[++r] = x;]
void dequeue( ) [if (f > r) cout << "underflow\n"; else cout
<< "Dequeue
Int peek ( ) [return f>r ? -1 : q(f); ]
void display ( ) [for (int i = f; i <= r; i++) cout << q[i]
<< ""; cout <<
int main ( ) [
    enqueue (10); enqueue (20); enqueue (30);
display ( );
cout << "front," " << peek ( ) << "\n";
dequeue ( );
display ( );
```

Output:

10 20 30

Fornt: 10

Dequeued: 10

20 30

2. Enqueue at the rear

```
#include <iostream>
using namespace std;

int main () {
    int q[5], r = -1;
    for (int x : {10, 20, 30}) {
        q[++r] = x;
        cout << "Inserted" << x << " at rear (index
        " << r << ")\n";
    }
}
```

Output:

Inserted 10 at rear (index 0)
Inserted 20 at rear (index 1)
Inserted 30 at rear (index 2)

Qu3. Dequeue from the front

```
#include <iostream>
using namespace std;
int main() {
    int q[] = {10, 20, 30}, f = 0, r = 2;
    while (f <= r) cout << "Deleted " << q[f++]
    " from front\n";
}
```

Output:

Deleted 10 from front

Deleted 20 from front

Deleted 30 from front

4 Peek (front without Deleting)

```
#include <iostream>
using namespace std;

int main () {
    int q[] = {10, 20, 30}, f=0, r=2;
    cout << "front element:" " << (f>r ? -1 : q[f]) <<
    "\n";
    cout << "front still; " << q[F] << "\n";
```

output;

front element; 10

front still : 10

Ques Display front to rear

```
#include <iostream>
using namespace std;
int main(){
    int q[] = {10,20,30,40}, f=0, r=3;
    cout << "front ->";
    for(int i=f; i<=r; i++) cout << q[i] <<
    cout << "<- rear\n";
```

output: front -> 10, 20 30 40 <- rear

Queue overflow

```
#include<iostream>
using namespace std;

int main() {
    int q[3], r=-1;
    for(int i=1; i<=4; i++) {
        if(r==2) cout<<"Queue overflow!
        cannot insert"<<i<<"\n";
        else q[++r]=i;
    }
}
```

output: Queue overflow! cannot insert 4

Qu7 Queue underflow

```
#include <iostream>
using namespace std;
int main() {
    int q[3], f = 0, r = -1;
    if (f > r) cout << "Queue underflow! Nothing to
    delete\n";
    else cout << q[f++];
}
```

Output: Queue underflow! Nothing to delete

48 Menu-Driven Queue

```
#include <iostream>
using namespace std;

int main(){
    int q[100], f=0, r=-1, ch, x;
    do {
    cout << "\n1. Enqueue 2. Dequeue 3. Peek 4. Display
                5. Exit ";

    cin>> ch;
    if (ch==1) {
      if (r==99) cout<< "overflow\n";
      else { cout << "value:"; cin >> x; q[++r]=x;}

    }
    else if (ch ==2) {
        if (f>r) cout << underflow\n";
    else cout <<"Dequeued: " <<q[f++] << "\n";

    }
    else if (ch==3) {
        if (f>r) cout << "empty\n";
        else cout<< "front: " << q[f] << "\n";
    }
```

```
else if (ch==4) {
    for (int i=f; i<=r; i++) cout << q[i] << " ";
    }
} while (ch!=5);
}
```

output;

1. Enqueue 2. Dequeue 3. peek 4. Display 5. Exit; 1
value: 10

1 Enqueue 2. Dequeue 3. peek 4. Display 5. Exit; 1
value; 20

1 Enqueue 2. Dequeue 3. peek 4. Display 5. Exit; 4
10 20

9. Count Elements in Queue

```
#include <iostream>
using namespace std;

int main() {
    int q[] = {10,20,30,40,50}, f=1, r=4;
    // one element already dequeued
    cout << "Elements in queue: " << (f>r ? 0:
    r-f+1) << "\n";
```

Output: elements in queue: 4

Q410 Search an Element

```
#include <iostream>
using namespace std;
int main() {
    int a[] = {10, 20, 30, 40}, f=0, l=3,
    cout << "search; "; cin>>key;
    for (int i=f; i<=l; i++)
    if (a[i] ==key {cout << "found at positi
    << i-f+1; return 0;}
    cout << "Not found";
```

output:

Search: 30
Found at position 3

11. Maximum and minimum

```
#include <bits/stdc++.h>
using namespace std;
int main () {
    int q[] = {30,10,50,20,40}, f=0, n=4;
    int mx = q[f], mn = q[f];
    for(int i = f + 1; i <= n; i++) mx = max
    (mx, q[i]), mn = min (mn, q[i])
    cout << "max: " << "\n min: " << mn;

}
```

output;

Max: 50
Min: 10

Qu12 Reverse a queue

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    queue<int> q;
    stack<int> st;
    for(int x : {10,20,30,40}) q.push(x);

while(!q.empty()) st.push(q.front()), q.pop();
    // queue -> stack

while(!st.empty()) q.push(st.top()), st.pop();
    // stack -> queue

cout << "reversed: ";
while(!q.empty()) cout << q.front() << " ", q.pop();
```

Output: Reversed: 40 30 20 10

13. Circular queue using array

```
#include <iostream>
using namespace std;
const int N = 4;
int q[N], f = 0, r = -1, cnt = 0;
void enqueue (int x) {
    if (cnt == N) { cout << "full\n"; return; }
    r = (r + 1) % N;
    q[r] = N;
    cent++;
}
void dequeue() {
    if (!cnt) { cout << "empty\n"; return }
    cout << "dequeued :" << q[F] << "\n";
    f = (f + 1) % N;
    cent --;
}
void display () { for (int i = 0; 0, i < cnt; i++) cout <<
    q[(f + i) % N] << "\n"; cout <<
int main () {
enqueue (10); enqueue (20); enqueue (30); enqueue
(40);
```

```
enqueue(50);                    //Full
dequeue(); dequeue();
enqueue(50); enqueue(60);       //reuses freed space
display();
}
```

Output

```
Full
Dequeued: 10
Dequeued: 20
30 40 50 60
```

Queue using linked list

```
#include <iostream>
using namespace std;

Struct node {int data; Node* next; };
Node* f = nullptr, *r = nullptr;

void enqueue (int x) {
        Node* n = new node {x, nullptr};
        if (!r) f = r = n;
        else r -> next = n, r = n;
}
void dequeue () {
        if (!f) {cout << "underflow\n"; return;}
        Node* t = f;
        cout << "dequeued: " << t -> data << "\n";
        f = f -> next;
        if (!f) r = nullptr;
        delete t;
}
int main () {
        enqueue (10); enqueue (20); enqueue (30);
        dequeue ();
        cout << "queue: ";
        for (Node* t = f; t = t -> next) cout << t -> data << (((
}
```

Output :

Dequeued : 10

Queue : 20 30

Linear vs Circular queue

```
#include <iostream>
using namespace std;
int main() {
    // linear queue (size 3)
    int q[3], f = 0, r = -1;
    for (int i = 1; i <= 3; i++) q[++r] = i;
    f += 2;
    cout << "linear: ";
    if (r == 2) cout << "overflow, 2 slots free but
    wasted\n";
    // circular queue (size 3)
    int c[3], r = 2, cnt = 3;
    cnt -= 2;                    // dequeue 2 elements
    cout << "circular: ";
    if (cnt < 3) { r = (r + 1) % 3; c[r] = 4;
    cout << "4 inserted at index " << r
```

output:

Linear: overflow, 2 slots free but wasted
Circular: 4 inserted at index 0